

Les arbres

1. Idée générale et analogie

Un arbre en informatique, c'est une façon d'organiser des données « en famille » : un élément de départ (la racine), qui peut avoir des « enfants », qui eux-mêmes peuvent avoir des enfants, etc.

Peupler ou explorer un arbre, c'est créer ou partir de la racine, puis créer ou suivre de proche en proche les liens qui mènent à d'autres éléments.

Exemples :

- dans la vie courante, les humains peuvent être placés dans un arbre généalogique (il y a un ancêtre commun au départ, tout en haut ; puis des branches qui se divisent en descendants) ;
- les fichiers d'un ordinateur sont organisés sous forme d'une structure arborescente de dossiers et (sous-)dossiers.

2. Définition et représentation

🎓 Arbre (structure de données *hiérarchique*)

Un arbre est :

- soit vide, que l'on représente parfois par le symbole $\emptyset$;
- soit il est constitué d'un nœud, appelé *racine* (qui « contient » une valeur), et de (sous-)arbres (qui sont eux-mêmes des arbres).

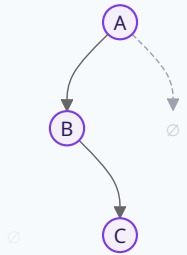
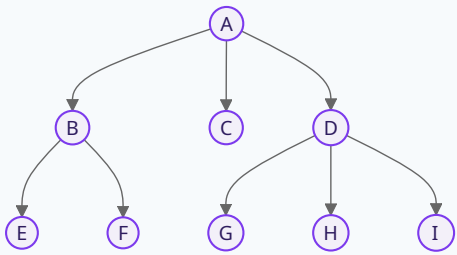
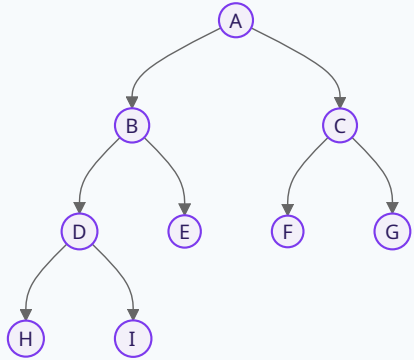
On appelle **enfants** d'un nœud **N** les *racines* de ses sous-arbres (ce sont donc des *nœuds*, pas des arbres). On dit réciproquement que **N** est le **parent** de ces nœuds.

💬 Remarque : est-ce la seule définition ? ▼

Non. En réalité, un **arbre** est un *cas particulier de graphe* (voir la fiche concernée). Cette définition **récursive** est la plus naturelle pour écrire des algorithmes en Python — on la rencontre souvent lors des parcours d'arbre.

🖍 Exemple : diverses représentations d'arbres ▼

Les définitions des notions de **taille** et **hauteur** figurent juste après.

Arbre vide	Arbre filiforme	Arbre quelconque	Arbre binaire complet
$\emptyset$ <i>Note : le plus souvent on ne dessine pas l'arbre vide</i>			
Hauteur : 0	Hauteur : 3	Hauteur : 3	Hauteur : 4
Taille : 0	Taille : 3	Taille : 9	Taille : 9

Remarque : précisions importantes ▼

- Chaque sous-arbre est *lui-même un arbre*, construit de la *même manière* — c'est ce qui rend la définition *récursive*.
- Tout nœud a donc *au moins* un sous-arbre : l'arbre vide. Mais il peut en avoir plus !
- Tout nœud a *au plus* un parent (le nœud racine n'en a pas ; tout *autre* nœud en a **exactement un**).
- Entre la racine et n'importe quel nœud, il existe *un seul chemin*, souvent appelé branche. Un arbre est donc *dépourvu de cycle* : on ne peut pas « sauter » d'une branche à une autre.
- L'ensemble des descendants d'un nœud est souvent *ordonné* (stocké dans une liste, par exemple).

✓ Vocabulaire associé aux propriétés d'un arbre

- **Feuille** : un nœud qui n'a pas de descendance.
- **Nœud interne** : nœud qui n'est pas une feuille.
- **Taille de l'arbre** : nombre de nœuds.
- **Profondeur d'un nœud** : mesure de l'éloignement entre un nœud et le nœud racine. C'est la longueur du chemin depuis la racine jusqu'à ce nœud (on compte le nombre de liaisons).
- **Hauteur de l'arbre** : nombre de nœuds sur le chemin le plus long depuis la racine jusqu'à une feuille. *Par convention*, l'arbre vide a **0** pour hauteur.
- **Chemin** : pour un nœud **N** donné, c'est la suite de nœuds qui mène de la racine **R** à ce nœud **N**. Pour tout nœud **N**, il existe un **unique** chemin menant de la racine **R** à ce nœud **N**.
- **Longueur d'un chemin** : c'est le nombre de liaisons présentes dans ce chemin.

⚠ Danger

La définition de la hauteur est sujette à *variations*, selon les besoins ou les habitudes.

L'arbre vide aura le plus souvent **0** pour hauteur : un arbre ayant *un seul nœud* aura donc une hauteur de **1**.

On peut toutefois rencontrer des situations où un arbre ayant un seul nœud aura une hauteur nulle. C'est un peu comme si le premier nœud (la racine) était « au niveau de la mer », le « niveau 0 ». Dans cette situation, d'aucuns considèrent que l'arbre vide a une hauteur de -1 , voire de $-\infty$.

✏ Exemple : hauteur d'un arbre vs longueur d'un chemin maximal ▼

- On convient que la hauteur de l'arbre vide est **0**.
L'exemple ci-dessus d'arbre binaire complet a pour hauteur **4**. Le nœud **H** est « le plus profond » de l'arbre : le chemin menant de **A** à **H** est $A \rightarrow B \rightarrow D \rightarrow H$, et sa longueur est **3**. Dans cette situation, la hauteur d'un arbre vaut 1 de plus que la longueur du plus long chemin.
- On convient que la hauteur d'un arbre d'un seul nœud est **0**.
Dans ce cas, la hauteur de cet arbre est **3** : la *même* valeur que celle de la longueur du plus long chemin.

⚠ RÈGLE ABSOLUE : bien lire le sujet, pour savoir quelle définition est choisie ⚠

3. Variantes importantes

3.1. Arbre binaire

Un **arbre binaire** est un arbre avec la contrainte que **chaque nœud a au plus deux enfants**. Les descendants (quand ils existent) sont nommés « **enfant gauche** » et « **enfant droit** ».

Remarque : précisions ▼

- Un enfant peut être absent : on parle alors de « sous-arbre gauche vide » ou « sous-arbre droit vide ».

- Pour abrégé, on écrit souvent « SAG » pour « sous-arbre gauche » et « SAD » pour « sous-arbre droit ».
- On retrouve la même définition *récursive* : un arbre binaire est soit vide, soit un nœud, un (sous-)arbre binaire gauche et un (sous-)arbre binaire droit (ces sous-arbres pouvant être vides).

✓ Encadrement de la taille en fonction de la hauteur

On compte la hauteur à partir de **0** pour l'*arbre vide* (donc un arbre à un nœud a une hauteur de **1**).

Pour un arbre binaire $\mathcal{A}$ de hauteur h , on a :

$$h \leq T(\mathcal{A}) \leq 2^h - 1$$

3.2. Arbre binaire de recherche

On définit un **arbre binaire de recherche** (souvent abrégé en **ABR** ou **BST** en anglais, pour *Binary Search Tree*) en ajoutant à la définition de base d'arbre binaire deux contraintes.

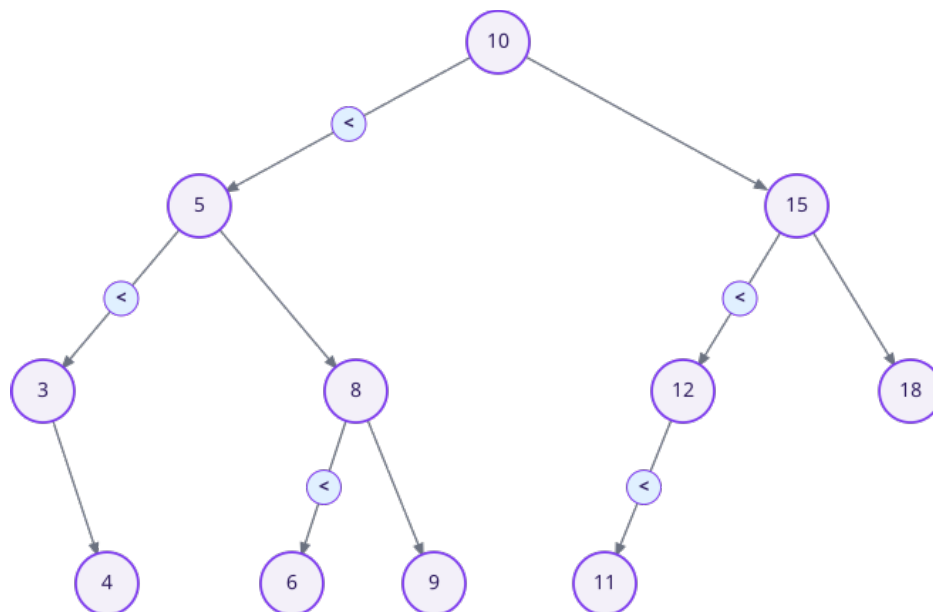
Pour tout nœud d'un arbre binaire de recherche :

- toutes les valeurs du *sous-arbre gauche* sont *inférieures* à la valeur du nœud ;
- toutes celles du *sous-arbre droit* sont *supérieures* à la valeur du nœud.

Remarque : doublons ou pas ? ▼

On rencontre souvent des ABRs *sans doublons* : impossible d'y trouver plusieurs fois la *même* valeur.

Exemple : un ABR ▼



- Considérons le nœud portant la valeur 4 : on a bien $3 < 4$, et 3 comme 4 sont *inférieurs* à 5, lui même *inférieur* à 10.
- De même, $6 < 8 < 9$, et ces 3 valeurs sont **supérieures** à 5, mais *inférieures* à 10.
- Enfin, $11 < 12 < 15$ mais 15, 11 et 12 sont **supérieurs** à 10.

Remarque : notation *potentiellement* ambiguë ▼

Positionner un label du type $<$ ou $>$ sur les liens est toujours problématique : dans quel sens faut-il suivre le lien — en descendant ou en remontant ? Ici, les symboles ne sont présents que sur les branches allant vers la gauche, et il faut lire globalement « de la gauche vers la droite », ce qui donne par exemple $3 < 5 < 10$.

4. Comment implémenter un arbre en Python ?

4.1. Approches basiques : listes et dictionnaires

Il existe de nombreuses façons d'utiliser listes et dictionnaires Python pour représenter un arbre. Les arbres *filiforme* et *quelconque* de l'[exemple précédent](#) peuvent être représentés par les listes suivantes (un arbre vide est ici représenté par une liste vide) :

```
filiforme = [ 'A', [ 'B', [ 'C', [] ] ] ]
quelconque = [ 'A', [ 'B', [ 'E', [] ],
                    [ 'F', [] ] ],
              [ 'C', [] ],
              [ 'D', [ 'G', [] ],
                    [ 'H', [] ],
                    [ 'I', [] ] ] ]
```

On peut aussi utiliser des **dictionnaires**, dont chaque clef est un nœud et chaque valeur une liste des descendants (on parle de *liste d'adjacence*, et on retrouve cette notion avec les **graphes**).

```
filiforme = { 'A' : [ 'B' ], 'B' : [ 'C' ], 'C' : [] }
quelconque = { 'A' : [ 'B', 'C', 'D' ],
               'B' : [ 'E', 'F' ],
               'C' : [],
               'D' : [ 'G', 'H', 'I' ],
               'G' : [],
               'H' : [],
               'I' : [] }
```

Cette forme de représentation souffre d'une légère incohérence : on n'implémente ni clairement ni proprement un *véritable* arbre vide, ici. En effet, un dictionnaire vide représente l'arbre vide, mais un sous-arbre est censé **être un arbre** ! Or, dans le code précédent, le sous-arbre de racine **C**, qui est un arbre-feuille (sans descendance), est représenté par... une *liste* vide !

De nombreuses autres représentations sont possibles ([voir ici](#) ou [là](#) par exemple). **Une représentation particulière, sous forme de liste, est néanmoins particulièrement intéressante, pour les arbres binaires :**

- le nœud racine de l'arbre se voit attribué l'indice 0 ;
- pour chaque nœud d'indice i , ses enfants reçoivent les indices $2i + 1$ et $2i + 2$;
- on initialise une liste Python à `[...] * (2**h - 1)` (on peut remplacer `...` par `None`), où h désigne la hauteur de l'arbre ;
- on place la valeur de chaque nœud d'indice i à la position i de cette liste.

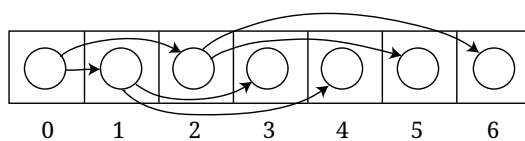


Schéma issu de Wikipedia

Exemple : transformation de l'ABR d'exemple en liste ▼

L'ABR montré [plus haut](#) a une hauteur de 4, et une taille de 11. On crée donc une liste Python de longueur $2^4 - 1 = 15$: `abr = [...] * 15`.

1. Le nœud racine est numéroté 0. Sa valeur 10 est affectée en position 0 de la *liste abr* : `abr[0] = 10`. On a alors `abr = [10, ..., ..., ..., ..., ..., ..., ..., ..., ..., ...]`
2. Son enfant gauche est numéroté $2 \times 0 + 1 = 1$. Sa valeur 5 est affectée en position 1 :
`abr = [10, 5, ..., ..., ..., ..., ..., ..., ..., ..., ..., ...]`
3. Son enfant droit est numéroté $2 \times 0 + 2 = 2$. Sa valeur 15 est affectée en position 2 :
`abr = [10, 5, 15, ..., ..., ..., ..., ..., ..., ..., ..., ..., ...]`
4. L'enfant gauche du nœud d'indice 1 (celui qui porte la valeur 5) est numéroté $2 \times 1 + 1 = 3$. Sa valeur 3 est affectée en position 3 :
`abr = [10, 5, 15, 3, ..., ..., ..., ..., ..., ..., ..., ..., ...]`
5. L'enfant droit du nœud d'indice 1 (celui qui porte la valeur 5) est numéroté $2 \times 1 + 2 = 4$. Sa valeur 8 est affectée en position 4 :
`abr = [10, 5, 15, 3, 8, ..., ..., ..., ..., ..., ..., ..., ..., ...]`
6. L'enfant gauche du nœud d'indice 2 (celui qui porte la valeur 15) est numéroté $2 \times 2 + 1 = 5$. Sa valeur 12 est affectée en position 5 :

```
abr = [10, 5, 15, 3, 8, 12, ..., ..., ..., ..., ..., ..., ..., ...]
```

7. L'enfant droit du nœud d'indice 2 (celui qui porte la valeur 15) est numéroté $2 \times 2 + 2 = 6$. Sa valeur 18 est affectée en position 6 :

```
abr = [10, 5, 15, 3, 8, 12, 18, ..., ..., ..., ..., ..., ..., ..., ...]
```

8. Et ainsi de suite. À la fin du processus, on obtient la liste :

```
abr = [10, 5, 15, 3, 8, 12, 18, ..., 4, 6, 9, 11, ..., ..., ...]
```

Noter que, dans ce cas, certaines cases restent à Ellipsis (...): elles correspondent à des nœuds inexistants.

✓ Retrouver le parent d'un nœud avec l'organisation en liste ▼

Soit un nœud d'emplacement i de la liste. La position de son parent (s'il existe) vaut $(i-1)//2$.

Remarque : condition d'optimalité ▼

Pour la culture : cette approche est *optimale* si l'arbre binaire est *complet*, car il n'y a aucune « perte de place ». On dit d'un arbre binaire qu'il est **complet** lorsque tous ses niveaux sont remplis, sauf (éventuellement) le dernier, qui doit être rempli *par la gauche*.

*Dans cette situation, la structure résultante est aussi appelée un **tas**.*

4.2. Approche POO

Il existe également de nombreuses façons d'implémenter les arbres binaires en Programmation Orientée Objet (POO). Voici l'exemple le plus fréquent, qui ne fait appel qu'à une seule classe, le plus souvent nommé `Noeud`.

```
class Noeud:
    def __init__(self, valeur):
        self.valeur = valeur
        self.gauche = None # sous-arbre gauche (None → SAG vide)
        self.droite = None # sous-arbre droit (idem)

    def inserer(self, valeur):
        """Insertion dans un ABR (sans doublons)."""
        if valeur < self.valeur:
            if self.gauche is None:
                self.gauche = Noeud(valeur)
            else:
                self.gauche.inserer(valeur) # Récursivité ici !
        elif valeur > self.valeur:
            # elif pour éviter les doublons
            if self.droite is None:
                self.droite = Noeud(valeur)
            else:
                self.droite.inserer(valeur) # Récursivité encore, ici...

    def rechercher(self, valeur):
        """Recherche dans un ABR ; retourne True/False."""
        if valeur == self.valeur:
            return True
        elif valeur < self.valeur:
            if self.gauche is None:
                return False
            else:
                # else est superflu, en réalité
                return self.gauche.rechercher(valeur) # Récursivité ici
        else:
            # Simplification du code précédent (pour recherche récursive à droite) :
            return self.droite is not None and self.droite.rechercher(valeur)
```

Utilisation :

```
racine = Noeud(10) # Création de la racine
for v in [5, 15, 3, 8, 12, 18, 4, 6, 9, 11]: # On obtient l'ABR d'exemple
    racine.inserer(v)
```

⚠ Difficulté avec cette approche !

La définition de la classe `Noeud` souffre d'un gros défaut : un arbre vide **n'est pas** de la classe `Noeud` : c'est `None`. Cela implique deux choses :

- pas moyen de stocker la valeur `None` dans un arbre ainsi implémenté ;
- le code est plus pénible à écrire. Il faut constamment vérifier si le (sous-)arbre vers lequel on souhaite poursuivre un traitement n'est pas vide. Concrètement, cela revient à vérifier s'il n'est pas `None`. Sans cette vérification, on risquerait d'appeler une méthode telle que `rechercher` ou `insérer` sur `None`. S'ensuivrait une erreur `AttributeError` car l'objet `None` ne dispose pas de ces méthodes.

Le problème fondamental avec cette approche est qu'un arbre vide n'est pas un arbre ! C'est juste `None` !

💬 Remarque : méthodes pour calculer hauteur et taille d'un arbre ▼

Il suffit d'ajouter à la classe `Noeud` les méthodes récursives suivantes.

```
def hauteur(self):                                # Approche récursive
    if self.gauche is None:
        h_gauche = 0
    else:
        h_gauche = self.gauche.hauteur()
    # Simplification du code précédent avec l'opérateur ternaire de Python
    h_droite = 0 if self.droite is None else self.droite.hauteur()
    return 1 + max(h_gauche, h_droite)

def taille(self):                                  # Approche récursive
    if self.gauche is None:
        t_gauche = 0
    else:
        t_gauche = self.gauche.taille()
    # Simplification du code précédent avec l'opérateur ternaire de Python
    t_droite = 0 if self.droite is None else self.droite.taille()
    return 1 + t_gauche + t_droite
```

5. Parcours d'arbre

Lorsque des informations sont organisées sous forme d'arbre, il faut pouvoir les retrouver, vérifier si une valeur est bien présente dans cet arbre, etc. Pour cela, on doit **parcourir** cet arbre. Il existe diverses approches : toutes ont pour objectif de visiter *tous les nœuds* selon un ordre précis.

- **Le parcours en largeur** est un parcours « niveau par niveau ». Dans le cadre de l'[exemple d'arbre binaire complet](#), au début de cette fiche, cela donnerait : $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I$.
- **Le parcours en profondeur** est un parcours qui vise à descendre immédiatement aussi profondément que possible, puis à reprendre l'exploration au dernier embranchement non complètement exploré. Il existe **3 variantes distinctes du parcours en profondeur**.

On a déjà effectué des parcours d'arbres, lors du calcul de la hauteur et de la taille d'un arbre : ces *parcours récursifs* sont les plus naturels. Mais des *parcours itératifs* sont aussi possibles : il faut pour cela **employer des structures de données auxiliaires, pile ou file**.

⚠ Rappel : point de vigilance !

En raison de l'implémentation retenue, il faut **toujours tester si un nœud existe AVANT** de l'explorer, avec `if noeud is None` ici. Autrement, on risque des erreurs ou des parcours impossibles à terminer correctement.

5.1. Parcours en largeur itératif

Le parcours en largeur n'est pas naturellement récursif : on n'évoquera donc que la version itérative. Quelle que soit l'approche, néanmoins, ce parcours nécessite une **file** comme structure de données intermédiaire. On va ici utiliser une liste Python « en mode file » (`fil.append(...)` pour enfiler, `fil.pop(0)` pour défiler).

```
def parcours_largeur(arbre):
    if arbre is not None:
        fil = [arbre]                # Liste qu'on utilise en « mode file » (FIFO)
        while fil != []:
            noeud = fil.pop(0)        # pop(0) pour défiler par la gauche
            print(noeud.valeur)
            if noeud.gauche is not None:
                fil.append(noeud.gauche) # append() pour enfiler par la droite
            if noeud.droite is not None:
                fil.append(noeud.droite) # idem
```

 **Remarque : pop(0) n'est pas très efficace ! ▼**

Le coût d'emploi d'une liste Python en mode file est assez médiocre : `pop(0)` s'exécute en $O(n)$! En effet, toutes les valeurs qui suivent celle qui sort de la file doivent « glisser vers la gauche »...

✓ Moyen mnémotechnique

Un parcours en Largeur utilise une file.

5.2. Parcours en profondeur préfixe

Ce parcours passe **d'abord par la racine**, puis par le sous-arbre gauche, puis par le sous-arbre droit. Il est naturellement récursif. **La version itérative utilise une *pile* comme structure de données auxiliaire.**

✓ Moyen mnémotechnique

Un parcours en Profondeur utilise une Pile.

```
def parcours_prefixe_recuratif(arbre):
    if arbre is None:
        return
    print(arbre.valeur)                # Traiter la racine d'abord
    parcours_prefixe_recuratif(arbre.gauche) # puis le sous-arbre gauche
    parcours_prefixe_recuratif(arbre.droite) # enfin le sous-arbre droit
```

La version itérative simule la récursivité avec une pile : il faut empiler d'abord le fils droit, puis le fils gauche, pour traiter le gauche en premier.

```
def parcours_prefixe_iteratif(arbre):
    if arbre is None:
        return
    pile = [arbre]                # Liste qu'on utilise en « mode pile » (LIFO)
    while pile:
        noeud = pile.pop()        # pop() pour dépiler par la droite (LIFO)
        if noeud is not None:     # Test utile, car...
            print(noeud.valeur)
            pile.append(noeud.droite) # ...ici on peut empiler None (à droite)
            pile.append(noeud.gauche) # ici aussi (mais None sera ignoré au tour suivant)
```

 **Remarque : pourquoi ajouter d'abord le sous-arbre de droite ? ▼**

Il est d'usage de progresser de la gauche vers la droite et, vu le mode de fonctionnement d'une pile (dernier arrivé, premier sorti), il faut empiler *d'abord* le SAD, pour que le SAG soit empilé *en dernier*. Ce sera ainsi le premier à être dépilé, et le SAG sera parcouru *avant* le SAD.

5.3. Parcours en profondeur infixe

Ce parcours passe **d'abord par le sous-arbre gauche**, *puis par la racine*, et enfin par le sous-arbre droit.

Le code récursif est très proche du précédent : il suffit de déplacer `print` entre les deux appels récursifs.

```
def parcours_infixe_recuratif(arbre):  
    if arbre is None:  
        return  
    parcours_infixe_recuratif(arbre.gauche) # Traiter le SAG d'abord  
    print(arbre.valeur)                    # puis la racine  
    parcours_infixe_recuratif(arbre.droite) # enfin le sous-arbre droit
```

 Remarque : pas de version itérative ici ! ▼

Une simple pile ne permet pas de différer le traitement d'un nœud jusqu'après l'exploration de ses descendants. Essayez de modifier le code itératif du parcours en profondeur préfixe pour vous en rendre compte... Il est *possible* mais *plus difficile* de faire un parcours en profondeur infixé itératif.

5.4. Parcours en profondeur postfixe (ou suffixe)

Ce parcours passe **d'abord par le sous-arbre gauche**, *puis par le sous-arbre droit*, et enfin par la racine.

Là encore, le code est quasiment identique aux codes précédents : il suffit de déplacer `print` après les deux appels récursifs.

```
def parcours_postfixe_recuratif(arbre):  
    if arbre is None:  
        return  
    parcours_postfixe_recuratif(arbre.gauche) # Traiter le SAG d'abord  
    parcours_postfixe_recuratif(arbre.droite) # puis le sous-arbre droit  
    print(arbre.valeur)                      # enfin la racine
```

 Remarque : pas de version itérative ici ! ▼

Ici encore, une simple pile ne permet pas de différer le traitement d'un nœud jusqu'après l'exploration de ses descendants. Essayez de modifier le code itératif du parcours en profondeur préfixe pour vous en rendre compte... Il est *possible* mais *plus difficile* de faire un parcours en profondeur postfixé itératif.

6. Liens avec d'autres notions

- La **récursivité** est naturelle pour les parcours d'arbres.
- Les **piles** servent à simuler les parcours en profondeur.
- Les **files** servent à faire des parcours en largeur.
- Les **graphes** généralisent la notion d'arbre.